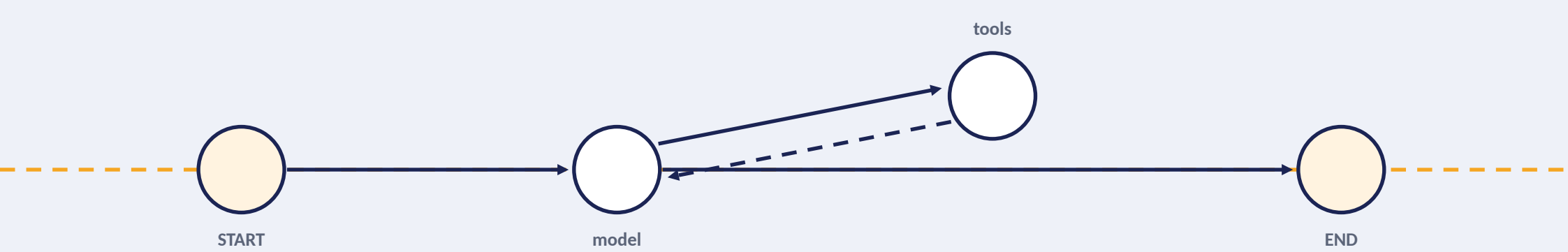
LANGGRAPH · CHAPTER 6 · FLASHCARDS

Tools and Tool Calling

The model proposes, you dispose

10 code cards on tools, tool calls and one round of execution



Asking is not running

bind_tools only advertises tools: the model replies with tool_calls and nothing runs

- The model picks a tool by its name and docstring, fills the args, and may ask for two at once.
- A question that needs no action gets a plain text answer and an empty tool_calls list.

PYTHON

```python
DEVICE_LOG: list[str] = []          # a tool body would write here if it ever ran

@tool
def turn_on_light(room: str) -> str:
    """Turn on the lights in one room of the house, e.g. 'kitchen'."""
    DEVICE_LOG.append(f"light on: {room}")
    return f"The {room} light is on."

# dim_lights, set_thermostat, lock_door: same shape

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
llm_with_tools = llm.bind_tools([turn_on_light, dim_lights,
                                 set_thermostat, lock_door])

response = llm_with_tools.invoke("I'm cold and it's dark in the living room")
print(repr(response.content))   # ''  <- no text when the model asks for tools
print(response.tool_calls)
# [{'name': 'set_thermostat', 'args': {'temperature': 22, 'unit': 'C'}, 'id': ...},
#  {'name': 'turn_on_light', 'args': {'room': 'living room'}, 'id': ...}]
print(DEVICE_LOG)               # []  <- nothing ran: the model only asked
```

The docstring is a prompt

@tool turns a function into a named, described tool the model can ask for

- No docstring fails at load time; a thin one like "Adds." builds fine and leaves the model guessing.
- The model proposes, your code disposes: every safety check lives on your side of that line.

PYTHON

```python
from langchain_core.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add a and b."""            # sent to the model as the tool description
    return a + b

# multiply, divide: same shape
tools = [add, multiply, divide]
llm_with_tools = llm.bind_tools(tools)     # advertise them to the model

add.invoke({"a": 3, "b": 4})               # 7: your code can still call it

@tool
def subtract(a: int, b: int) -> int:       # no docstring
    return a - b
# ValueError: Function must have a docstring if description not provided.

@tool(description="Subtract b from a.")    # or pass the description explicitly
def subtract(a: int, b: int) -> int:
    return a - b
```

What the model actually sees

A name, a description and a JSON schema: that is the tool's whole API to the model

- Say when NOT to use the tool, and name its domain ("the guide"), not its mechanism ("FAISS").
- ge/le become minimum/maximum for the model and are enforced by Pydantic before the body runs.

PYTHON

```python
class SearchDocsInput(BaseModel):
    query: str = Field(
        description="A focused search phrase, e.g. 'prompt injection defenses'. "
        "Rewrite the user's question into keywords; do not paste it verbatim.")
    k: int = Field(
        default=3, ge=1, le=5,
        description="How many passages to return (1-5). "
        "Use more for broad questions.")

# What bind_tools sends, printed with convert_to_openai_tool (abridged):
print(json.dumps(convert_to_openai_tool(search_docs), indent=2))
# {"type": "function", "function": {
#    "name": "search_docs",
#    "description": "Search the internal 'LLM Production ... Guide'. ... Do NOT
#        use it for general knowledge, arithmetic, or current events.",
#    "parameters": {"properties": {
#       "query": {"type": "string", "description": "A focused search ..."},
#       "k": {"type": "integer", "default": 3, "minimum": 1, "maximum": 5, ...}},
#     "required": ["query"]}}}
```

A tool that closes over the index

A factory takes the prebuilt index and returns a tool that only reads it

- content_and_artifact: the tool returns a pair, text for the model and data for your code.
- An empty search returns a message, not an exception: exceptions are for things that went wrong.

PYTHON

```python
def make_search_tool(vectorstore):
    """Build a search tool that closes over an already-built index."""

    @tool(args_schema=SearchDocsInput, response_format="content_and_artifact")
    def search_docs(query: str, k: int = 3):
        """Search the internal 'LLM Production Security and Observability Guide'.
        ... Do NOT use it for general knowledge, arithmetic, or current events."""
        docs = vectorstore.similarity_search(query, k=k)
        if not docs:
            # "Nothing found" is an answer, not an exception: tell the model.
            return "No relevant passages found in the guide.", []
        text = "\n\n".join(f"[Chunk {i + 1}]\n{doc.page_content}"
                           for i, doc in enumerate(docs))
        sources = [{"chunk": i + 1,
                    "source": Path(doc.metadata.get("source", "?")).name,
                    "preview": doc.page_content[:60].replace("\n", " ")}
                   for i, doc in enumerate(docs)]
        return text, sources        # (content for the model, artifact for your code)

    return search_docs

search_docs = make_search_tool(build_vectorstore(DOCS_DIR))   # Chapter 5's index
```

Propose, execute, return

Invoke a tool with the whole tool-call dict and you get back a ToolMessage with its id

- The model wrote its own queries and split the question into two calls in one reply.
- content goes back to the model; artifact stays on the message for your code and is never sent.

PYTHON

```python
# Step 2: the model PROPOSES
proposal = llm.bind_tools([search_docs]).invoke([
    SystemMessage(SYSTEM_PROMPT),
    HumanMessage("What is prompt injection and how do we defend against it?"),
])
proposal.content       # ''
proposal.tool_calls    # two calls, queries written by the model:
#   search_docs(query='prompt injection definition', k=1)
#   search_docs(query='prompt injection defenses', k=1)

# Step 3: your code EXECUTES
call = proposal.tool_calls[0]
tool_message = search_docs.invoke(call)    # the whole dict -> a ToolMessage
tool_message.tool_call_id == call["id"]    # True: it points at its request
tool_message.content     # '[Chunk 1]\nLLM01 — Prompt Injection is ...'
tool_message.artifact    # [{'chunk': 1, 'source': 'llm_production_guide.txt'...

search_docs.invoke(call["args"])    # args only -> a str: no id, no artifact
```

What a tool node does

Look the tool up by name, run it with the model's args, reply with a ToolMessage

- Copying tool_call["id"] into tool_call_id is what lets the model match result to request.
- ToolNode does the same, plus error handling and concurrent runs; read this once, then use it.

PYTHON

```python
from langchain_core.messages import ToolMessage
from langgraph.graph import MessagesState

tools_by_name = {t.name: t for t in tools}

def tool_node(state: MessagesState):
    """Run every requested tool; wrap each result in a ToolMessage."""
    result = []
    last_message = state["messages"][-1]
    for tool_call in getattr(last_message, "tool_calls", []):
        t = tools_by_name[tool_call["name"]]       # look up by name
        observation = t.invoke(tool_call["args"])  # run with the model's args
        result.append(ToolMessage(content=str(observation),
                                  tool_call_id=tool_call["id"]))  # copy id
    return {"messages": result}      # add_messages appends them to history

builder.add_node("tools", tool_node)   # wired like any other node
```

One round of tools in a graph

ToolNode runs the requested tools; tools_condition routes to "tools" or to END

- tools_condition reads the last message: tool calls go to "tools", a plain answer ends the run.
- answer uses the model with no tools bound, so after one round it can only reply. Ch 7 adds the loop.

PYTHON

```python
@tool
def get_today() -> str:
    """Return today's weekday and date, e.g. 'Wednesday, 30 September 2026'.

    Use it whenever the user asks what day or date it is today.
    """
    return date.today().strftime("%A, %d %B %Y")

llm_with_tools = llm.bind_tools([get_today])  # call_llm: may REQUEST get_today
                                              # answer: plain llm, no tools bound
builder = StateGraph(MessagesState)
builder.add_node("llm", call_llm)
builder.add_node("tools", ToolNode([get_today]))        # must be named "tools"
builder.add_node("answer", answer)
builder.add_edge(START, "llm")
builder.add_conditional_edges("llm", tools_condition)   # "tools" or END
builder.add_edge("tools", "answer")                     # forward only: one round
builder.add_edge("answer", END)

# "What day of the week is it today?"   human -> ai -> tool -> ai
# "What is the capital of Japan?"       human -> ai
```

Pair results by tool_call_id

One AIMessage can carry several calls; ToolNode runs them concurrently, one reply each

- Each ToolMessage carries the id of the call it answers: pair results by id, never by position.
- Parallel calls are the model's choice; bind_tools(..., parallel_tool_calls=False) turns them off.

PYTHON

```python
# A one-node graph: START -> "tools" (ToolNode(smart_home_tools)) -> END
model_reply = AIMessage(content="", tool_calls=[    # written by hand: no API key
    {"name": "set_thermostat", "args": {"temperature": 22, "unit": "C"},
     "id": "call_A"},
    {"name": "turn_on_light", "args": {"room": "living room"}, "id": "call_B"},
])
messages = graph.invoke({"messages": [
    HumanMessage("I'm cold and it's dark in the living room"), model_reply,
]})["messages"]

requests = {call["id"]: call
            for msg in messages if isinstance(msg, AIMessage)
            for call in msg.tool_calls}
for msg in messages:
    if isinstance(msg, ToolMessage):
        call = requests[msg.tool_call_id]       # look up the request by id
        print(f"{msg.tool_call_id}: {call['name']} -> {msg.content}")
# call_A: set_thermostat -> Thermostat set to 22.0°C.   (float in the schema)
# call_B: turn_on_light -> The living room light is on.
# DEVICE_LOG: ['thermostat: 22.0C', 'light on: living room']  <- they really ran
```

Three kinds of trouble

Bad args go back to the model, transient failures are retried, empty results are answers

- By default ToolNode returns only argument errors to the model and re-raises everything else.
- A retry re-runs the whole tools node: fine for searches, risky for "send email" or "charge card".

PYTHON

```python
# 1. Bad arguments: returned to the model as a ToolMessage with status="error"
#    the model asked for search_docs(query="x", k=9):
#   Error invoking tool 'search_docs' with kwargs {'query': 'x', 'k': 9} with error:
#     k: Input should be less than or equal to 5
#     Please fix the error and try again.

# 2. Transient failures: re-raised by ToolNode, so a RetryPolicy can catch them
from langgraph.types import RetryPolicy

builder.add_node("tools", ToolNode(tools),
                 retry_policy=RetryPolicy(max_attempts=3, retry_on=ConnectionError))
# waits 0.5 s, then 1 s (doubling, with jitter) before attempts 2 and 3

# 3. Empty results are not errors: search_docs returns
#    ("No relevant passages found in the guide.", [])

# Choose which exceptions reach the model:
ToolNode(tools, handle_tool_errors=True)      # every exception -> error ToolMessage
ToolNode(tools, handle_tool_errors="Search failed, try a narrower query.")
```

Three ways tools bite

An unknown tool name, a retry that repeats work, and a tool node with the wrong name

- A hand-written node trusts the model's tool name; ToolNode answers a bad name with an error message.
- Keep tools with side effects out of retried nodes, and name the ToolNode "tools" for tools_condition.

PYTHON

```python
# 1. Listing 6.6's tool node indexes the dict with the model's tool name
t = tools_by_name[tool_call["name"]]  # model asks for "subtract" -> KeyError, crash
# ToolNode replies instead, as a ToolMessage with status="error":
#   "Error: subtract is not a valid tool, try one of [add, multiply, divide]."

# 2. RetryPolicy re-runs EVERY call in the turn, not just the one that failed
# AIMessage asks for lookup(...) and flaky(...); flaky raises ConnectionError once
# after the retry: runs == {'lookup': 2, 'flaky': 2}   <- lookup ran twice

# 3. tools_condition returns the string "tools"
builder.add_node("search", ToolNode(tools))              # any other name...
builder.add_conditional_edges("llm", tools_condition)
builder.compile()
# ValueError: At 'llm' node, 'tools_condition' branch found unknown target 'tools'

# Checked on langgraph 1.2.6 with hand-written AIMessages, no model calls
```